

Inverse Estimation of Aerodynamic Parameters from Observed Projectile Trajectories

Atharv Maheshkumar Tirthkar

1. Introduction

1.1 Origin

First of all, I would like you to go through section 5 of this paper and then proceed with 1.1.

My interest in this project began when I studied kinematics in the ninth grade. At that stage, I learned the standard equations of projectile motion and accepted them as mathematical tools for solving problems. Later, while studying fluid mechanics in the eleventh grade, I came across Stokes' Law, which describes the viscous drag experienced by an object moving through a fluid. Along with concepts such as buoyant force, this introduced me to the idea that the motion of a real object is governed by forces that are often more complex than those considered in idealized classroom models.

This raised an important question in my mind: if the trajectory of a projectile can be observed experimentally, is it possible to work backwards and estimate the physical parameters responsible for that motion? In other words, can the mathematical model itself be reconstructed from observations rather than assumed beforehand?

To explore this question, I combined concepts from physics, mathematics, and computer programming to develop a computational model capable of estimating aerodynamic parameters directly from experimentally observed projectile trajectories. Beyond obtaining numerical results, this project became an opportunity to understand how mathematical modelling and scientific computation can be used to investigate real physical systems.

1.2 Previous Works

Projectile motion is frequently modelled using idealized equations that neglect aerodynamic effects or assume standard drag models. In several research works, time-position data are extracted from video recordings and subsequently analysed through computational simulation. Some of the relevant studies are summarized below.

[1] *Projectile Motion: Experimental Datasets and Classroom Exercises* (Fabian B. Wadsworth *et al.*, 2025) discusses polynomial fitting and mathematical modelling using experimentally obtained trajectory data, and compares the reconstructed trajectories with ideal projectile models that neglect air resistance.

[2] *Modeling the Effect of Air Drag on Basketball Trajectories: An Analysis of Trajectory Deviation, Drag Coefficient, and Optimal Launch Angle* by Haohang Li, Zheng Wei, Haohan Pu, and Aditya Vishwakarma formulates an optimization problem to study the influence of air drag on basketball trajectories. The work employs a polynomial model for trajectory fitting and presents a graphical analysis of trajectory deviation, drag coefficient, and optimal launch angle.

[3] *A Case Study on Projectile Motion* by Vivek S., Rohini R., and Dhanya G. formulates a quadratic drag model and compares its predictions with those of an ideal no-drag model. The study reports a small deviation between the experimentally observed and analytically computed values across seven trajectory parameters.

1.3 About the project

This project investigates whether a physically consistent model of projectile motion can be reconstructed directly from experimentally observed trajectory data. Beginning with time-stamped position measurements obtained through motion analysis, numerical differentiation is used to estimate velocity and acceleration, while piecewise Hermite interpolation is implemented to reconstruct a smooth approximation of the trajectory.

A generalized projectile model incorporating aerodynamic drag and ambient wind is then formulated as a system of coupled ordinary differential equations. The unknown aerodynamic parameters – including drag coefficient, drag exponent, and wind velocity components – are estimated through nonlinear optimization by minimizing the discrepancy between the simulated and experimentally observed trajectories.

The complete computational framework has been implemented in Python using scientific computing libraries such as NumPy and SciPy. The estimated model is finally validated by comparing its predicted trajectory with the experimentally reconstructed trajectory using normalized root mean square deviation (NRMSD) as a quantitative measure of agreement.

2. The Workflow

2.1 Getting our data

The data sample concerned to us can be collected in different number of ways whichever you choose. However, the method I will suggest is to use is:

- i. Use any motion capturing device such as a Mobile Camera, DSLR etc. to get our projectile motion video sample. Follow 2.1 (ii) on how to use *Tracker*.
- ii. *Tracker* is officially distributed in two formats –
 - a. Web Application
 - b. Downloadable and Executable Application

Now, we have the video with us. Transfer it to a computer so we can work with it. For getting the coordinates of the projectile in each frame, we will use ‘Tracker’ – a project by *Open Source Physics (OSP)*.

Information on its installation, usage, terms and conditions are available at – <https://opensourcephysics.github.io/tracker-website>

For our needs, I will suggest the later format (b) as it requires no internet and can handle large video files significantly more comfortably than the Web version.

- iii. Save timestamps and coordinates data (following 2.1 (i) to a text file in the format shown below:
time_stamp_1 x_coordinate_1 y_coordinate_1
time_stamp_2 x_coordinate_2 y_coordinate_2
time_stamp_3 x_coordinate_3 y_coordinate_3
...
- iv. There are exactly *three* columns and any number of rows depending upon the recorded frames, representing the nomenclature as used and make sure there is no any other text present other than the above format to ensure proper working of our program.

**Additional Instructions/Tips:*

- a. Make sure you have a reasonable contrast in video so as to differentiate comfortably – the ball/stone (projectile) from its surroundings.
- b. Try to record the video at highest FPS (Frames Per Second) setting available in the device, which will deliver a higher accuracy in our program.
- c. The recorded projectile trajectory should be confined to a plane parallel to the plane of the camera lens. In other words, efforts should be made to ensure that the projectile has minimal motion toward or away from the lens. Reason being *Tracker's* capabilities – it is not designed for three-dimensional analysis.

2.2 Initializing Computation

As we have position information of projectile in different frames. We calculate velocity and acceleration of the same using fundamental principles of differentiation focusing majorly on method of central differences to minimize error in the calculations concerned.

The coordinate data we have collected will most probably gaps where we have no information about the projectile. For example, suppose there are two successive time stamps $t = t_1$ and $t = t_2$, we only have the projectile position data at those points but not in the time interval between those two points. When we plot such a data, it is going to most probably look rough, spiky and maybe uneven too. To make it smooth, we use piecewise Hermite interpolation and implement it over the entire dataset.

2.3 The Model and its Optimization

Our objective is to estimate aerodynamic parameters – Wind Velocity and Drag Coefficient. We design and use the system of equations as the basis to program our model. Continuous fitting of set of parameters and minimizing the RMS (Root Mean Square) deviation of our model with actual

experimentally derived dataset yields us the best fit parameter set which will be our final result.

3. Mathematical Formulation

3.1 Differentiation

Method of central differences serves as a more accurate method than Euler method of differentiation. We will be implementing the method of central differences over the dataset of n data points; except for the places where we cannot i.e. initial and final points.

For example, suppose for i^{th} point, we can say,

$$v_{x,i} = \frac{x_{i+1} - x_{i-1}}{t_{i+1} - t_{i-1}}, v_{y,i} = \frac{y_{i+1} - y_{i-1}}{t_{i+1} - t_{i-1}}$$

But for initial point, we use forward difference $v_{x,1} = \frac{x_2 - x_1}{t_2 - t_1}$, similarly for $v_{y,1}$.

For final point, we use backward difference so, $v_{x,n} = \frac{x_n - x_{n-1}}{t_n - t_{n-1}}$, similarly for

$v_{y,n}$.

Similarly for acceleration.

3.2 Aerodynamics: Formulation of Equations

We consider the projectile to be planar, constrained only to xy plane at any time t , hence we have $v_z = 0$ (constant).

We define the following (assuming in stationary ground frame unless specified):

- i. $\vec{v}$ = Velocity of Centre Of Mass (COM) of object
- ii. $\vec{v}_w$ = Velocity of wind (assumed constant over the concerned experiment)
- iii. $\vec{v}_{rel}$ = Velocity of COM relative to the air = $\vec{v} - \vec{v}_w$
- iv. $\vec{a}$ = Acceleration of centre of mass of object
- v. b = Drag Coefficient
- vi. k = Index of Drag

$$\text{Drag Force acting on COM} = \vec{F}_{drag} = -b|\vec{v}_{rel}|^k \hat{v}_{rel}$$

where,

- $\vec{v} = v_x \hat{i} + v_y \hat{j}$
- $\vec{v}_w = v_{xw} \hat{i} + v_{yw} \hat{j} + v_{zw} \hat{k}$
- $\vec{v}_{rel} = \vec{v} - \vec{v}_w = (v_x - v_{xw}) \hat{i} + (v_y - v_{yw}) \hat{j} - v_{zw} \hat{k}$
- $\hat{v}_{rel} = \frac{\vec{v}_{rel}}{|\vec{v}_{rel}|}; |\vec{v}_{rel}| = \sqrt{(v_x - v_{xw})^2 + (v_y - v_{yw})^2 + v_{zw}^2}$

$$\vec{F}_{drag} = -b|\vec{v}_{rel}|^k \frac{\vec{v}_{rel}}{|\vec{v}_{rel}|} = -b|\vec{v}_{rel}|^{k-1}\vec{v}_{rel}$$

$$\text{So, net force on COM} = \vec{F} = -mg\hat{j} - b|\vec{v}_{rel}|^{k-1}\vec{v}_{rel}$$

According to Newton's Second Law, we have,

$$\Rightarrow \vec{F} = m\vec{a} = -mg\hat{j} - b|\vec{v}_{rel}|^{k-1}\vec{v}_{rel}$$

$$\Rightarrow \vec{a} = -g\hat{j} - \frac{b}{m}|\vec{v}_{rel}|^{k-1}\vec{v}_{rel}$$

$$\Rightarrow a_x\hat{i} + a_y\hat{j} + a_z\hat{k}$$

$$= -g\hat{j} - \frac{b}{m}|\vec{v}_{rel}|^{k-1} \left((v_x - v_{xw})\hat{i} + (v_y - v_{yw})\hat{j} - v_{zw}\hat{k} \right)$$

$$\therefore \frac{dv_x}{dt}\hat{i} + \frac{dv_y}{dt}\hat{j} + \frac{dv_z}{dt}\hat{k}$$

$$= - \left(\frac{b}{m}|\vec{v}_{rel}|^{k-1}(v_x - v_{xw}) \right) \hat{i}$$

$$- \left(\left(g + \frac{b}{m}|\vec{v}_{rel}|^{k-1} \right) (v_y - v_{yw}) \right) \hat{j} + \left(\frac{b}{m}|\vec{v}_{rel}|^{k-1}v_{zw} \right) \hat{k}$$

From above discussion, we finally arrive at the following system of *six* first order coupled ODEs (Ordinary Differential Equations):

- i. $\frac{dx}{dt} = v_x$
- ii. $\frac{dy}{dt} = v_y$
- iii. $\frac{dz}{dt} = v_z = 0$
- iv. $\frac{dv_x}{dt} = a_x = -\frac{b}{m}|\vec{v}_{rel}|^{k-1}(v_x - v_{xw})$
- v. $\frac{dv_y}{dt} = a_y = -g - \frac{b}{m}|\vec{v}_{rel}|^{k-1}(v_y - v_{yw})$
- vi. $\frac{dv_z}{dt} = a_z = \frac{b}{m}|\vec{v}_{rel}|^{k-1}v_{zw}$

3.3 Piecewise Quintic Hermite Interpolation

As we have already discussed, sole aim of this interpolation is to make projectile curve smooth for plotting. Consider two points 1 and 2, where we have complete motion of projectile i.e. time and position.

Treating the projectile's position (x, y) as functions of t in time interval between 1 and 2, we assume,

$$x = f(t) \text{ and } y = g(t).$$

Instead of directly assuming functions f and g to be linear, we want both of the equations to: correctly pass through points 1 and 2, have a naturally aligned steepness, have right concavity. To ensure these conditions are met, we have the following set of equations to be followed at points 1 and 2:

Sr. No.	Parent Equation: $x = f(t)$	Parent Equation: $y = g(t)$
1.	$f(t_1) = x_1$	$g(t_1) = y_1$

2.	$f(t_2) = x_2$	$g(t_2) = y_2$
3.	$f'(t_1) = v_{x1}$	$g'(t_1) = v_{y1}$
4.	$f'(t_2) = v_{x2}$	$g'(t_2) = v_{y2}$
5.	$f''(t_1) = a_{x1}$	$g''(t_1) = a_{y1}$
6.	$f''(t_2) = a_{x2}$	$g''(t_2) = a_{y2}$

As there are six unique derived equations for each of the two assumed parent equations, we can compute $x = f(t)$ and $y = g(t)$ by setting *six* unknown coefficients to each of the equations. Therefore, the assumed equations along with their first and second derivatives (with respect to t) transform into polynomial equations:

- $f(t) = a_{0x} + a_{1x}t + a_{2x}t^2 + a_{3x}t^3 + a_{4x}t^4 + a_{5x}t^5$
- $g(t) = a_{0y} + a_{1y}t + a_{2y}t^2 + a_{3y}t^3 + a_{4y}t^4 + a_{5y}t^5$
- $f'(t) = a_{1x} + 2a_{2x}t + 3a_{3x}t^2 + 4a_{4x}t^3 + 5a_{5x}t^4$
- $g'(t) = a_{1y} + 2a_{2y}t + 3a_{3y}t^2 + 4a_{4y}t^3 + 5a_{5y}t^4$
- $f''(t) = 2a_{2x} + 6a_{3x}t + 12a_{4x}t^2 + 20a_{5x}t^3$
- $g''(t) = 2a_{2y} + 6a_{3y}t + 12a_{4y}t^2 + 20a_{5y}t^3$

To solve such a system of *twelve* equations (as in the table), we formulate two matrix equations:

$$1. \begin{bmatrix} 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 1 & t_2 & t_2^2 & t_2^3 & t_2^4 & t_2^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 1 & 2t_2 & 3t_2^2 & 4t_2^3 & 5t_2^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \\ 0 & 0 & 2 & 6t_2 & 12t_2^2 & 20t_2^3 \end{bmatrix} \times \begin{bmatrix} a_{0x} \\ a_{1x} \\ a_{2x} \\ a_{3x} \\ a_{4x} \\ a_{5x} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ v_{x1} \\ v_{x2} \\ a_{x1} \\ a_{x2} \end{bmatrix} \text{ or } XA_x = Y_x$$

$$2. \begin{bmatrix} 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 1 & t_2 & t_2^2 & t_2^3 & t_2^4 & t_2^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 1 & 2t_2 & 3t_2^2 & 4t_2^3 & 5t_2^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \\ 0 & 0 & 2 & 6t_2 & 12t_2^2 & 20t_2^3 \end{bmatrix} \times \begin{bmatrix} a_{0y} \\ a_{1y} \\ a_{2y} \\ a_{3y} \\ a_{4y} \\ a_{5y} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ v_{y1} \\ v_{y2} \\ a_{y1} \\ a_{y2} \end{bmatrix} \text{ or } XA_y = Y_y$$

where,

$$\begin{aligned}
\text{A. } X &= \begin{bmatrix} 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & t_1^5 \\ 1 & t_2 & t_2^2 & t_2^3 & t_2^4 & t_2^5 \\ 0 & 1 & 2t_1 & 3t_1^2 & 4t_1^3 & 5t_1^4 \\ 0 & 1 & 2t_2 & 3t_2^2 & 4t_2^3 & 5t_2^4 \\ 0 & 0 & 2 & 6t_1 & 12t_1^2 & 20t_1^3 \\ 0 & 0 & 2 & 6t_2 & 12t_2^2 & 20t_2^3 \end{bmatrix} \\
\text{B. } A_x &= \begin{bmatrix} a_{0x} \\ a_{1x} \\ a_{2x} \\ a_{3x} \\ a_{4x} \\ a_{5x} \end{bmatrix} \\
\text{C. } A_y &= \begin{bmatrix} a_{0y} \\ a_{1y} \\ a_{2y} \\ a_{3y} \\ a_{4y} \\ a_{5y} \end{bmatrix} \\
\text{D. } Y_x &= \begin{bmatrix} x_1 \\ x_2 \\ v_{x1} \\ v_{x2} \\ a_{x1} \\ a_{x2} \end{bmatrix} \\
\text{E. } Y_y &= \begin{bmatrix} y_1 \\ y_2 \\ v_{y1} \\ v_{y2} \\ a_{y1} \\ a_{y2} \end{bmatrix}
\end{aligned}$$

As we have computed the function for interval 1 – 2, we repeat the same process of solving these matrix equations and getting a function for all intervals over entire dataset.

4. Computational Implementation

We have finally arrived at the stage where we have all our pre-requisites and need to actual program a software in Python to turn our abstract equations into a form of physical significance.

4.1 Libraries and Modules

As said earlier, libraries used herein are –

- i. NumPy – For handling numerical datasets, arrays, matrix equations etc.

- ii. SciPy (Modules: a. integrate, b. optimize) – *integrate* is used for solving the system of coupled ODEs and *optimize* for minimizing the fitting error.
- iii. Matplotlib (Module: pyplot) – For plotting trajectory and error plots.
- iv. OpenPyXL (Module: Workbook) – For storing processed data and results in an excel sheet.

We begin our program by importing these libraries:

```
import numpy as np

from scipy import integrate

from scipy import optimize

import matplotlib.pyplot as plt

from openpyxl import Workbook
```

4.2 Embedding experimentally derived data

We have the required data in the specific format we need, so we extract the timestamp and position datasets.

```
#extracting coordinates

loc = input("\nLocation of dataset: ")

m = float(input("Mass of particle (in kg): "))

g = 9.81

ds = np.loadtxt(loc)

t_list = ds[:, 0]

x_list = ds[:, 1]

y_list = ds[:, 2]

size = len(t_list)

print("\nComputation Initialized...")
```

4.3 Initiating the computation

We are now equipped with raw time-position data, which we will use the method as discussed in 3.1 to write our program further. After the above computational work, we also define a set of variables that refer to initial fixed parameters (position and velocity) of concerned projectile.

```
#computing velocities and accelerations at given points
```

```
vx_list = []
```

```
vy_list = []
```

```
ax_list = []
```

```
ay_list = []
```

```
#forward difference - velocity
```

```
dt = t_list[1] - t_list[0]
```

```
vx_list.append((x_list[1] - x_list[0]) / dt)
```

```
vy_list.append((y_list[1] - y_list[0]) / dt)
```

```
#central differences - velocity
```

```
for i in range(1, size - 1):
```

```
    dt = t_list[i + 1] - t_list[i - 1]
```

```
    vx_list.append((x_list[i + 1] - x_list[i - 1]) / dt)
```

```
    vy_list.append((y_list[i + 1] - y_list[i - 1]) / dt)
```

```
#backward difference - velocity
```

```
dt = t_list[-1] - t_list[-2]
```

```
vx_list.append((x_list[-1] - x_list[-2]) / dt)
```

```
vy_list.append((y_list[-1] - y_list[-2]) / dt)
```

```
#forward difference - acceleration
```

```
dt = t_list[1] - t_list[0]
```

```

ax_list.append((vx_list[1] - vx_list[0]) / dt)

ay_list.append((vy_list[1] - vy_list[0]) / dt)


#central differences - acceleration
for i in range(1, size - 1):

    dt = t_list[i + 1] - t_list[i - 1]

    ax_list.append((vx_list[i + 1] - vx_list[i - 1]) / dt)

    ay_list.append((vy_list[i + 1] - vy_list[i - 1]) / dt)


#backward difference - acceleration
dt = t_list[-1] - t_list[-2]

ax_list.append((vx_list[-1] - vx_list[-2]) / dt)

ay_list.append((vy_list[-1] - vy_list[-2]) / dt)


vx_list = np.array(vx_list)

vy_list = np.array(vy_list)

ax_list = np.array(ax_list)

ay_list = np.array(ay_list)


#initial parameters - position and velocities

x0 = x_list[0]

y0 = y_list[0]

vx0 = vx_list[0]

vy0 = vy_list[0]

```

In this snippet as well as in the later ones, we do not initially declare an array as a NumPy array rather a normal Python array but after doing work on that array, we then convert it into NumPy array. It is so because, everytime we append a NumPy array, new memory space is allocated each time which slows down the computation. Although it is negligible for smaller datasets, it may

show up when dataset is large. On the other hand, Python arrays do not allocate new memory space each time we append it, it therefore imparts a lower processing overhead.

4.4 Formulation: Piecewise Hermite Interpolation

The principles and methods used for programming the interpolation is discussed precisely in section 3.3. The aim of this particular program section is to generate new 98 data points (which are supposed to be equidistant in terms of timestamps and contain position data at each of those points) between two given data points. So, there will be total 100 points including the already known two data points.

```
#piecewise hermite interpolation

def hermite(t1, t2, x1, x2, y1, y2,
            vx1, vx2, vy1, vy2,
            ax1, ax2, ay1, ay2):

    X = np.array([
        [1, t1, t1**2, t1**3, t1**4, t1**5],
        [1, t2, t2**2, t2**3, t2**4, t2**5],
        [0, 1, 2*t1, 3*t1**2, 4*t1**3, 5*t1**4],
        [0, 1, 2*t2, 3*t2**2, 4*t2**3, 5*t2**4],
        [0, 0, 2, 6*t1, 12*t1**2, 20*t1**3],
        [0, 0, 2, 6*t2, 12*t2**2, 20*t2**3]
    ], dtype=float)

    Y_forx = np.array([x1, x2, vx1, vx2, ax1, ax2], dtype=float)
    Y_fory = np.array([y1, y2, vy1, vy2, ay1, ay2], dtype=float)

    A_forx = np.linalg.solve(X, Y_forx)
    A_fory = np.linalg.solve(X, Y_fory)
```

```

t_in = np.linspace(t1, t2, 100)

x_in = np.zeros_like(t_in)

y_in = np.zeros_like(t_in)

for i in range(6):

    x_in += A_forx[i] * t_in**i

    y_in += A_fory[i] * t_in**i

return t_in, x_in, y_in

```

This returns a tuple containing three datasets in form of lists: t_in , x_in and y_in . There are 100 elements in each of those three lists which make the curve between points 1 and 2 much smoother than it was before because additional 98 points have been added between initially available data points which successfully marks the aim of this interpolation.

4.5 Implementing the piecewise interpolation globally

We have the soul for making the interpolation over entire data sample we have derived through the experiment, which is the formulation of idea of interpolation. The work needed to advance it further is to make this piecewise interpolation go global, meaning, implementing the interpolation technique we have developed over the entire dataset.

```

#implementing interpolation over entire dataset

x_interp_all = []

y_interp_all = []

t_interp_all = []

for i in range(size - 1):

    t_seg, x_seg, y_seg = hermite(

        t_list[i], t_list[i+1],

        x_list[i], x_list[i+1],

        y_list[i], y_list[i+1],

        vx_list[i], vx_list[i+1],

```

```

        vy_list[i], vy_list[i+1],

        ax_list[i], ax_list[i+1],

        ay_list[i], ay_list[i+1]
    )

    if i != size - 2:

        t_seg = t_seg[:-1]

        x_seg = x_seg[:-1]

        y_seg = y_seg[:-1]

    t_interp_all.extend(t_seg)

    x_interp_all.extend(x_seg)

    y_interp_all.extend(y_seg)

t_interp_all = np.array(t_interp_all)

x_interp_all = np.array(x_interp_all)

y_interp_all = np.array(y_interp_all)

print("\nStage 1/3 Complete - Experimental trajectory processed.")

```

4.6 Equations governing the Projectile's Trajectory

The equations of motion of projectile are exactly the *six* equations ($i - vi$) formulated in 3.2. We define a function *proj* with arguments: t , *state*, wx , wy , wz , m where x , y , vx , vy = *state*. *state* has been defined as such because the parameters it holds define state of the projectile at each point of time. Although, there is absolutely no problem if the arguments in *state* are written explicitly with other parameters.

To avoid any cases of division by zero in future (if program extends), we put a safety on v_mag as well as on $vrel_mag$ as written in the program.

```

#equations of the projectile

vz0 = 0

```

```

def proj(t, state, b, k, wx, wy, wz, m):

    x, y, vx, vy, vz = state

    vrelx = vx-wx

    vrel_y = vy-wy

    vrel_z = -wz

    vrel_mag = max(np.sqrt(vrelx**2 + vrel_y**2 + vrel_z**2), 1e-12)

    dxdt = vx

    dydt = vy

    dvxdt = -(b/m)*(vrel_mag**(k-1))*vrelx

    dv_ydt = -g-((b/m)*(vrel_mag**(k-1))*vrel_y)

    dv_zdt = -(b/m)*(vrel_mag**(k-1))*vrel_z

    return dxdt, dydt, dvxdt, dv_ydt, dv_zdt

```

4.7 Objective Function

The objective of this project is to estimate the unknown aerodynamic parameters by minimizing the deviation between the experimentally observed trajectory and the trajectory predicted by our mathematical model. To accomplish this, we define an objective function, which serves as the quantity to be minimized by the optimization algorithm.

The parameters to be estimated are the drag coefficient (b), drag exponent (k), and the three components of the ambient wind velocity (w_x , w_y , w_z). For every parameter set proposed by the optimizer, the system of coupled ordinary differential equations is solved using the numerical method discussed previously. The resulting simulated trajectory is then compared with the experimentally observed trajectory at the corresponding time instants.

The discrepancy between the two trajectories is quantified using the Root Mean Square (RMS) deviation named as *error*, defined as

$$error = \sqrt{\frac{1}{N} \sum_{i=1}^N [(x_i - x_i^{fit})^2 + (y_i - y_i^{fit})^2]}$$

where N denotes the total number of experimental observations, (x_i, y_i) are the experimentally observed coordinates, and (x_i^{fit}, y_i^{fit}) are the corresponding coordinates predicted by the model.

The optimization process continuously updates the parameter values until the objective function attains its minimum value, thereby obtaining the set of aerodynamic parameters that best reproduces the experimentally observed projectile trajectory. RMS deviation is chosen because it provides a single quantitative measure of the overall fitting error while assigning greater weight to larger deviations, making it suitable for trajectory fitting.

The corresponding implementation of the objective function in Python is shown below.

```
#solving coupled ordinary differential equations
```

```
def objective(guess):  
    b, k, wx, wy, wz = guess  
  
    solution = integrate.solve_ivp(  
        proj,  
        (t_list[0], t_list[-1]),  
        [x0, y0, vx0, vy0, vz0],  
        t_eval=t_list,  
        args=(b, k, wx, wy, wz, m),  
        method="DOP853",  
        rtol=1e-10,  
        atol=1e-12  
    )
```

```
    if not solution.success:
```

```
        return np.inf
```

```
    x_fit = solution.y[0]
```

```
    y_fit = solution.y[1]
```

```

error = np.sqrt(
    np.mean((x_list - x_fit)**2 + (y_list - y_fit)**2)
)

return error

```

4.8 Optimization

We have input for SciPy's minimizer ready – which is the *objective* function defined above. The method used for minimization is *L-BFGS-B* (Limited-memory Broyden–Fletcher–Goldfarb–Shanno with Bound constraints). *L-BFGS-B* was selected because it efficiently handles bounded nonlinear optimization problems while requiring relatively little memory. Since this method requires initial guess and bounds as well for the variables, we set them as herein. For wind velocity bounds, we set maximum magnitude per direction as 120 m/s because highest non-tornadic wind speed ever recorded was roughly 113 m/s (reference: https://en.wikipedia.org/wiki/Wind_speed)

Initial guess:

$b = 0.05, k = 2, wx = wy = wz = 0$

Bounds:

$b \in [0,10] \equiv (0, 10); k \in [0,5] \equiv (0, 5); wx, wy, wz \in (-120, 120) \equiv (\text{None}, \text{None})$

As handled the case of failure of ODE solver, likewise, we do for optimizer.

```
#optimize for best fit values of b, k, wx, wy, wz
```

```
initial_guess = [0.05, 2, 0, 0, 0]
```

```
result = optimize.minimize(
```

```
    objective,
```

```
    initial_guess,
```

```
    method="L-BFGS-B",
```

```
    bounds=[
```

```
        (0, 10),
```

```
        (0, 5),
```

```
        (-120, 120),
```

```

        (-120, 120),
        (-120, 120)
    ]
)

if not result.success:

    raise RuntimeError(result.message)

b_fit, k_fit, wx_fit, wy_fit, wz_fit = result.x

print("\nStage 2/3 Complete - Aerodynamic parameters estimated.")

```

4.9 The Fitted Curve

Once the optimization algorithm has successfully estimated the best-fitting aerodynamic parameters, the mathematical model is solved once again. During the optimization process, the system of coupled ordinary differential equations is evaluated only at the original experimental time instants, since these are sufficient for computing the objective function.

However, the interpolated experimental trajectory contains 98 additional points between every pair of consecutive observations. To enable a direct visual comparison between the fitted model and the interpolated trajectory, the system of ordinary differential equations is solved again using the estimated parameters, but this time at the same set of interpolated time instants.

Apart from replacing the original time vector with the interpolated one, the numerical solution procedure remains unchanged. Since the solution is now evaluated at a much larger number of time instants, it produces a smooth and finely sampled trajectory, commonly referred to as a *dense solution*. This dense trajectory is subsequently used for plotting and validating the fitted model against the interpolated experimental trajectory.

The corresponding implementation is shown below.

```

solution = integrate.solve_ivp(
    proj,
    (t_list[0], t_list[-1]),
    [x0, y0, vx0, vy0, vz0],

```

[illegible]

(-wz_fit)**2)**(k_fit-1))*(-
wz_fit)

4.10 Reporting and Saving Results

Having obtained both the interpolated experimental trajectory and the fitted trajectory, the performance of the estimated model is evaluated quantitatively. Since the two trajectories contain the same number of corresponding data points at identical time instants, the deviation between them can be computed directly.

The Root Mean Square (*RMS*) deviation between the interpolated and fitted trajectories is given by

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N [(x_i^{int} - x_i^{fit})^2 + (y_i^{int} - y_i^{fit})^2]}$$

where N denotes the total number of corresponding observations, (x_i^{int}, y_i^{int}) are the interpolated experimental coordinates, and (x_i^{fit}, y_i^{fit}) are the corresponding coordinates predicted by the fitted model.

To make this error independent of the overall size of the trajectory, the *RMS* deviation is normalized using the trajectory length L . The trajectory length is computed numerically using the discrete equivalent of the standard curve-length integral,

$$L = \int_{x_1}^{x_2} \sqrt{1 + \left(\frac{dy}{dx}\right)^2} dx \approx \sum_{i=1}^{N-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$$

The final quantity reported is the Percent Normalized Root Mean Square Deviation (*%NRMSD*), defined as

$$\%NRMSD = \frac{RMS}{L} \times 100\% = \frac{\sqrt{\frac{1}{N} \sum_{i=1}^N [(x_i^{int} - x_i^{fit})^2 + (y_i^{int} - y_i^{fit})^2]}}{L} \times 100\%$$

A validation check is also performed before reporting the estimated parameters. If the estimated drag coefficient (b) is extremely close to zero (approximately 10^{-6} or smaller), it is inferred that the experimental data does not contain sufficient aerodynamic information to reliably estimate the drag exponent and wind velocity components. In such cases, these parameters are not reported.

Finally, the estimated aerodynamic parameters, trajectory length, *%NRMSD*, and the complete interpolated and fitted trajectory datasets are displayed to the

user and saved in an Excel workbook. The Excel file allows convenient storage, analysis, and comparison of results, and is created using the OpenPyXL library.

```
#trajectory length

trajectory_length = np.sum(np.hypot(np.diff(x_interp_all),
np.diff(y_interp_all)))

percent_norm_rms_dev = (np.sqrt(np.mean((x_interp_all -
x_fit_dense)**2 + (y_interp_all - y_fit_dense)**2)) /
trajectory_length) * 100

print("\nStage 3/3 Complete - Best-fit trajectory generated and
model validated.")

#saving results

wb = Workbook()

ws = wb.active

ws.title = "Interpolated Trajectory Data"

# Trajectory data header

ws.append([

    "t [s]",

    "x [m]",

    "y [m]",

    "z [m]",

    "vx [m/s]",

    "vy [m/s]",

    "vz [m/s]",

    "ax [m/s^2]",

    "ay [m/s^2]",
```

```

        "az [m/s^2]"
    ])

# Trajectory data
for i in range(len(t_interp_all)):
    ws.append([
        t_interp_all[i],
        x_fit_dense[i],
        y_fit_dense[i],
        "-",
        vx_fit_dense[i],
        vy_fit_dense[i],
        "-",
        ax_fit_dense[i],
        ay_fit_dense[i],
        "-"
    ])

# Estimated parameters in columns L and M
ws["L1"] = "Estimated Model Parameters"
ws["M1"] = "Value"

ws["L2"] = "Drag coefficient (b)"
ws["M2"] = b_fit

ws["L3"] = "Drag exponent (k)"
ws["M3"] = k_fit

```

```

ws["L4"] = "Wind velocity (wx) [m/s]"

ws["M4"] = wx_fit

ws["L5"] = "Wind velocity (wy) [m/s]"

ws["M5"] = wy_fit

ws["L6"] = "Wind velocity (wz) [m/s]"

ws["M6"] = wz_fit

ws["L7"] = "Wind speed [m/s]"

ws["M7"] = np.sqrt(wx_fit**2 + wy_fit**2 + wz_fit**2)

ws["L8"] = "Trajectory length [m]"

ws["M8"] = trajectory_length

ws["L9"] = "% Normalized RMS deviation "

ws["M9"] = percent_norm_rms_dev

wb.save("interpolated_trajectory_data.xlsx")

print("\nData Saved Successfully.")

#reporting results

print("\nAssuming drag force =  $-b \cdot (|v_{rel}|^k) \hat{v}_{rel}$ :")

if b_fit <= 1e-6:

    print(f"\nb = {b_fit:.6f}")

```

```

        print("\nNo measurable drag was detected")

        print("Therefore, the drag exponent (k) and wind velocity are
not identifiable from this dataset.")

    else:

        print(f"b = {b_fit:.6f}")

        print(f"k = {k_fit:.6f}")

        print(f"Wind Velocity in x = {wx_fit:.4f} m/s")

        print(f"Wind Velocity in y = {wy_fit:.4f} m/s")

        print(f"Wind Velocity in z = {wz_fit:.4f} m/s")

        print(f"\nWind Speed: {np.sqrt(wx_fit**2 + wy_fit**2 +
wz_fit**2):.4f} m/s")

    print(f"\nNormalized RMS Deviation of Best Fit Model =
{percent_norm_rms_dev:.4f} %")

    print(f"Trajectory Length = {trajectory_length:.2f} m")

```

4.11 Plots

The computational procedure is now complete, and both the interpolated experimental trajectory and the best-fit trajectory predicted by the model have been obtained. To visually assess the quality of the fitting, these trajectories are plotted on the same graph.

In addition to the trajectory comparison, the pointwise deviation between the interpolated experimental trajectory and the fitted trajectory is computed at each corresponding time instant and plotted as a function of time. This plot provides a visual representation of how the fitting error varies throughout the projectile's flight and complements the normalized RMS deviation reported in the previous section.

The corresponding implementation is shown below.

```

view_plot = input("\nPress 'Enter' to view trajectory plots and
deviation plot for the Best Fit Model and Interpolated
Experimentally Observed data... ")

#plots

plt.figure(1)

```

```

plt.plot(x_list, y_list,
label="experimentally_observed_trajectory", color="green")

plt.plot(x_fit_dense, y_fit_dense, label="best_fit_trajectory",
color="red")

plt.plot(x_interp_all, y_interp_all,
label="interpolated_expt_trajectory", color="blue")


plt.xlabel("x [m]")

plt.ylabel("y [m]")

plt.title("Projectile Trajectories")

plt.legend()

plt.grid(True)


#plotting deviation

plt.figure(2)


deviation = np.hypot(

    x_interp_all - x_fit_dense,

    y_interp_all - y_fit_dense

)


plt.plot(

    t_interp_all,

    deviation,

    color="black"

)


plt.xlabel("time [s]")

```

```
plt.ylabel("Deviation [m]")

plt.title("Normalized RMS Deviation: Fitted Model vs. Interpolated
Experimental Trajectory\n")

plt.grid(True)

#showing the plots

plt.show()

print("\n*Program has been terminated succesfully.\n")
```

5. Further Advancements

5.1 Why

The development of this project has instilled in me several valuable skills and habits, including independently learning advanced topics in mathematics, numerical analysis, and computer science, particularly error and exception handling, as well as the effective use of scientific computing libraries such as NumPy and SciPy. These disciplines formed the core foundation of this project and significantly contributed to its successful completion.

I believe this project does not mark an end, but rather the beginning of a broader pursuit of embedding computation into physical phenomena to extract deeper mathematical insight into the world around us.

5.2 What

Since the current project primarily focuses on the estimation of aerodynamic parameters of an actual projectile, several topics remain worthy of further investigation. Exploring these areas would help transform the present implementation into a more robust, stable, and generalized software framework. Some possible directions for future work are:

- i. Program an independent software for analysis of three-dimensional projectiles instead of restricting to a two-dimensional system, like an advanced *Tracker*. Furthermore, this will also help handle z directional projectile's components much more comfortably than we are currently able to. This naturally raises a question of how a three-dimensional projectile should be captured experimentally. Interesting.
- ii. Embedding Computer Vision (CV) functionality into the program to automatically detect and analyse projectile motion directly from raw video footage, representing a natural extension of section 5.2 (i). This would eliminate the dependence on external software such as *Tracker*, making the entire workflow self-contained.

- iii. Comparing and validating the proposed model against existing theoretical formulations, such as Stokes' Law, the Magnus effect, and Lord Rayleigh's drag equation.
- iv. Studying the effects of variable wind velocities on projectile motion which is assumed to be constant for concerned experiment in this version. Although this would introduce additional mathematical and computational complexities, it would significantly improve the realism and applicability of the model.
- v. Prediction of projectile's parameters using the trajectory data available up to that point. Such a capability could eventually be extended to applications such as real-time trajectory prediction and projectile range optimization.
- vi. Focusing on improving the numerical analysis part of the program, especially numerical differentiation. Since differentiation, particularly second-order differentiation, amplifies the noise present in experimental data and forms the basis of almost all the computations that follow, making this stage more stable is of great importance. Similarly, improving the numerical conditioning of the optimization process would help reduce overflow and convergence issues. I believe these are some of the most fundamental areas that require further work.

6. Feedback and Correspondence

Constructive feedback, suggestions, and discussions related to this project are always welcome. For any queries, comments, or correspondence regarding this work, I may be contacted at:

Atharv Maheshkumar Tirthkar
E-mail: atharvtirthkar@gmail.com